

Package ‘cryptocompareR’

October 28, 2023

Title Cryptocompare Api in R

Version 1.0.0

Author Beniamino Sartini

Maintainer Beniamino Sartini <sartinibeniamino@gmail.com>

Description A wrapper for different informations regarding the crypto market, such 'OHLCV' time series, social data, exchange data and much more.

License MIT

Encoding UTF-8

LazyData true

Depends R (>= 3.5.0),

Imports lubridate,
dplyr,
purrr,
magrittr,
stringr,
tidyr (>= 1.0.0),

Suggests broom,
knitr,
rmarkdown,
testthat (>= 2.1.0),

RoxygenNote 7.1.1

Roxygen list(markdown = TRUE)

VignetteBuilder knitr

R topics documented:

add_EMA	2
add_EMA.data.frame	2
as_unix	3
cc_all_exchanges	4
cc_blockchain_balance	4
cc_blockchain_historical	5
cc_eth_staking	6
cc_exchange_historical	7
cc_general_info	8

cc_L1_order_book	9
cc_orderbook_exchange	10
cc_order_book_L2	10
cc_price_historical	11
cc_rate_limit	12
cc_set_api_key	12
cc_set_environment	13
cc_social_historical	13
cc_symbol_id	14
cc_symbol_price	14
cryptocompare_api	15
EMA	16
SMA	16
Index	18

add_EMA	<i>add_EMA</i>
---------	----------------

Description

generic method for compute the Exponential Moving Average of an object.

Usage

add_EMA(object, ...)

Arguments

... other parameters.

add_EMA.data.frame	<i>add_EMA.data.frame</i>
--------------------	---------------------------

Description

method for the class of objects data.frame for computing the Exponential Moving Average and adding to the dataset in a tidy way.

method for the class of objects data.frame for computing the Simple Moving Average and adding to the dataset in a tidy way.

Usage

```
## S3 method for class 'data.frame'
add_EMA(object, variable = "Close", n = 10, smooth = 2, na.rm = TRUE)

## S3 method for class 'data.frame'
add_SMA(object, variable = "Close", n = 10, na.rm = TRUE)
```

Arguments

variable	the name of the variable of which we would compute the moving average.
n	integer, the number of length of the moving average.
smooth	numeric, a parameter to use for smoothing the moving average.
na.rm	logical,

Value

a tibble with one more column named as EMA_ + variable, _n.

a tibble with one more column named as SMA_ + variable, _n.

Examples

```
df <- data.frame(x = runif(1000, 0, 10))
df <- add_EMA(df, variable = "x", n = 10)

df <- data.frame(x = runif(1000, 0, 10))

# the new column will be named as "SMA_x_10"
df <- add_EMA(df, variable = "x", n = 10)
```

as_unix

as_unix

Description

This function convert a Date into Unix format: an integer denoting the number of seconds from the origin date, usually "1970-01-01".

Usage

```
as_unix(x = NULL, milliseconds = FALSE, origin = "1970-01-01")
```

Arguments

x	character or Date, in the format "yyyy-mm-dd" or "dd-mm-yyyy". In alternative it is possible to specify the Date also with the time in the format "yyyy-mm-dd hh:mm:ss" or "dd-mm-yyyy hh:mm:ss"
milliseconds	logical, if TRUE the output will be converted in milliseconds, otherwise it will be in seconds.
origin	character denoting the origin date.

Value

integer character

Examples

```
as_unix("2023-01-01")
as_unix("01-01-2023")
as_unix("2023-01-01 10:23:23")
as_unix("01-01-2023 10:23:23")
```

cc_all_exchanges	<i>cc_all_exchanges</i>
------------------	-------------------------

Description

retrieve the names of all the exchanges available in cryptocompare Api.

Usage

```
cc_all_exchanges()
```

Value

character vector

Examples

```
cc_all_exchanges()
```

cc_blockchain_balance	<i>cc_blockchain_balance</i>
-----------------------	------------------------------

Description

historical balance for Bitcoin blockchain only. The output will be a tibble containing the total Bitcoin held and the number of wallet by different amounts range. In order to access to this endpoint a free api key is needed.

Usage

```
cc_blockchain_balance(from = NULL, to = NULL, api_key = NULL)
```

Arguments

from	character or Date, the start date for importation.
to	character or Date, the end date for importation.
api_key	character

Details

The output of the function will be a tibble with 6 columns:

- Date: the observation date.
- Symbol: the symbol of interest.
- From: the minimum amount of Bitcoin held by some wallets.
- To: the maximum amount of Bitcoin held by some wallets.
- Amount: the total number of Bitcoin held by wallets having a quantity of Bitcoin between From and To.
- Wallets: the total number of Wallets having a quantity of Bitcoin between From and To.

Value

tibble

Examples

```
# Api key is required
## Not run:
yourapikey <- "yourapikey"
cc_blockchain_balance(from = "2017-01-01",
                      to = "2023-01-01",
                      api_key = yourapikey)

## End(Not run)
```

```
cc_blockchain_historical
cc_blockchain_historical
```

Description

Blockchain historical data, such as the number of transactions, active wallets or mining rewards are difficult to retrieve in a readable format. Cryptocompare simplify substantially the procedure of download, aggregate and summaries the data for each day. With this function it is possible to obtain the historical information for most of the blockchain. The data available for the Bitcoin's blockchain starts from 2017. In order to access to this function a free api key is needed.

Usage

```
cc_blockchain_historical(
  symbol = NULL,
  start = NULL,
  end = NULL,
  api_key = NULL
)
```

Arguments

symbol	character, the symbol of interest.
start	character or Date, the start date for importation.
end	character or Date, the end date for importation.
api_key	character

Value

A tibble.

Examples

```
# Api key is required
## Not run:
yourapikey <- "yourapikey"

cc_blockchain_historical(symbol = "BTC", start = "2017-01-01",
                        end = "2023-01-01", api_key = yourapikey)

cc_blockchain_historical(symbol = "ETH", start = "2017-01-01",
                        end = "2023-01-01", api_key = yourapikey)

cc_blockchain_historical(symbol = "DOGE", start = "2017-01-01",
                        end = "2023-01-01", api_key = yourapikey)

## End(Not run)
```

cc_eth_staking

Staking Rate Historical Day

Description

Ethereum staking interest rate, from when the Beacon chain was active.

Usage

```
cc_eth_staking(start = NULL, end = NULL, api_key = NULL)
```

Arguments

start	character or Date, the starting date for importation.
end	character or Date, the end date for importation.
api_key	character

Details

The Ether staking rate is powered by Attestant, a company dedicated to the business of staking. They provide a non-custodial Ethereum 2 managed staking service, giving customers the ability to stake their Ether using Attestant's infrastructure while always retaining full control of their assets. You can only use this endpoint with a valid api_key. The Ether rate is generally updated daily at 11:00 am UTC for the previous day.

Value

tibble

Examples

```
# Api key is required
## Not run:
yourapikey <- "yourapikey"
cc_eth_staking(start = "2022-01-01", end = "2023-01-01", api_key = yourapikey)
cc_eth_staking(start = "2021-01-01", end = "2022-01-01", api_key = yourapikey)
cc_eth_staking(start = "2023-01-01", end = "2023-01-19", api_key = yourapikey)

## End(Not run)
```

cc_exchange_historical

cc_exchange_historical

Description

cc_exchange_historical

Usage

```
cc_exchange_historical(
  symbol = NULL,
  start = NULL,
  end = NULL,
  exchange = NULL,
  currency = "USD",
  interval = c("daily", "hourly"),
  api_key = NULL
)
```

Arguments

symbol	character, the symbol of interest.
start	character or Date, the start date for importation.
end	character or Date, the end date for importation.
exchange	character, the exchange in which retrieve the information, if NULL it will be returned a mean between all the exchanges and denoted with "global".
currency	character, the currency in which convert the output.
interval	character, the frequency of the data, it can be "daily", "hourly".
api_key	character

Value

a tibble, with 7 columns (Date, Symbol, Currency, Exchange, VolumeTo, VolumeFrom, Volume)

cc_general_info

*Cryptocompare General Info***Description**

Retrieve the general information, using the cryptocompare Api.

Usage

```
cc_general_info(
  info = "coins",
  currency = "USD",
  api_key = NULL,
  verbose = FALSE
)
```

Arguments

info	Character. Without an api_key it is possible to access only to "coins" and "exchanges" information. Using a free api key, it is possible to access to all the endpoints: • "blockchain": returns a list of all coins for which we currently get blockchain data from IntoTheBlock. • "cards": returns general info about all the integrated crypto cards. • "coins": returns general info about all the integrated coins. • "companies": returns general info about all the integrated mining companies. • "contracts": returns general info about all the integrated mining contracts. • "equipment": returns general info about all the integrated mining equipment. • "exchanges": returns general info and 24h volume for all integrated exchanges. • "gambling": returns general info about all the integrated exchanges. • "wallets": returns general info about all the integrated wallets. • "pools": returns general info about all the integrated mining pools.
currency	Character. Currency in which convert the monetary values.
api_key	Character, optional. Api key from cyptocompare.
verbose	Logical. If TRUE it will display warning messages if you do not insert a valid api key for the api-key-only endpoints.

Value

a tibble

Examples

```
# Endpoints reachable without an Api Key
cc_general_info("coins")
cc_general_info("exchanges")

# Endpoints reachable only with a valid Api Key
## Not run:
api_key <- "yourapikey"
cc_general_info("gambling", api_key = api_key)
cc_general_info("wallets", api_key = api_key)
cc_general_info("cards", api_key = api_key)
cc_general_info("contracts", api_key = api_key)
cc_general_info("companies", api_key = api_key)
cc_general_info("equipment", api_key = api_key)
cc_general_info("pools", api_key = api_key)
cc_general_info("blockchain", api_key = api_key)

## End(Not run)
```

cc_L1_order_book

Order Book L1 Top

Description

Returns latest order book Level 1 bid/ask values for the specified symbol, currency and exchange.

Usage

```
cc_L1_order_book(symbol,
                  currency,
                  exchange = NULL,
                  api_key = NULL,
                  quiet = FALSE)
```

Arguments

symbol	Character,
currency	Character
exchange	Character
api_key	Character, mandatory.
quiet	Logical

Details

It is possible to access to this endpoint with an `api_key`.

cc_orderbook_exchange	<i>Exchanges With Order Book Data</i>
-----------------------	---------------------------------------

Description

Returns all the exchanges we have order book level 2 data.

Usage

```
cc_orderbook_exchange(api_key = NULL)
```

Arguments

api_key	Character, mandatory.
---------	-----------------------

Details

It is possible to access to this endpoint with an api_key.

cc_order_book_L2	<i>Order Book L2 Snapshot</i>
------------------	-------------------------------

Description

Returns latest order book Level 2 data snapshot for the requested exchange.

Usage

```
cc_order_book_L2(api_key = NULL)
```

Arguments

api_key	Character, mandatory.
---------	-----------------------

Details

It is possible to access to this endpoint with an api_key.

cc_price_historical *cc_price_historical*

Description

Get the Open, High, Low, Close, Volume, Open-Close change (OC) and High-Low change (HL) for the specified symbol.

Usage

```
cc_price_historical(
  symbol,
  start = NULL,
  end = NULL,
  exchange = NULL,
  currency = "USD",
  interval = c("daily", "hourly", "minutely"),
  api_key = NULL,
  verbose = TRUE
)
```

Arguments

symbol	A string or character vector containing the symbols of interest. (e.g. "BTC", "ETH", "BNB",...)
start	A Date or character vector containing the start date for importation.
end	A Date or character vector containing the end date for importation.
exchange	A string or character vector containing the exchange to use, if NULL it will be returned a mean between all the exchanges and denoted with "global".
currency	A string or character vector containing the currency in which convert the output (e.g. "USD", "USDT", "BNB", "EUR",...).
interval	A string or character vector containing the frequency of the data, it can be "daily", "hourly" or "minutely". The access to minutely data is restricted to the last 7 days, even with the free api key.
api_key	character containing your api key.
verbose	logical

Value

tibble, with 12 columns (Date, Symbol, Currency, Exchange, High, Low, Open, Volume, Close, Adj, OC, HL)

Examples

```
# Daily Data from General Exchange for the Pair BTC-USD
cc_price_historical(symbol = "BTC", start = "2022-01-01", end = "2023-01-01",
  exchange = NULL, currency = "USD", interval = "daily")

# Hourly Data from General Exchange for the Pair BTC-USD
cc_price_historical(symbol = "BTC", start = "2022-01-01 00:00:00", end = "2023-01-01 00:00:00",
```

```

exchange = NULL, currency = "USD", interval = "hourly")

# Daily Data from Binance Exchange for the Pair BTC-BUSD
cc_price_historical(symbol = "BTC", start = "2022-01-01", end = "2023-01-01",
                    exchange = "Binance", currency = "BUSD", interval = "daily")

# Daily Data from Kucoin Exchange for the Pair BTC-USDT
cc_price_historical(symbol = "BTC", start = "2022-01-01", end = "2023-01-01",
                    exchange = "Kucoin", currency = "USDT", interval = "daily")

```

cc_rate_limit	<i>Cryptocompare Rate limit</i>
---------------	---------------------------------

Description

Find out how many calls you have left in the current month, day, hour, minute and second

Usage

```
cc_rate_limit(hour = FALSE)
```

Arguments

hour	Logical. Default is FALSE. If TRUE will return the number of calls done and left in the current hour.
------	---

cc_set_api_key	<i>cc_set_api_key</i>
----------------	-----------------------

Description

add a personal api key to the "cryptocompare" environment in such a way to avoid to inserting it manually in every function's call where it is needed.

function to use the api key saved in the "cryptocompare" environment. In order to add it you can use the function cc_set_api_key

Usage

```
cc_set_api_key(api_key = NULL, quiet = FALSE)
```

```
cc_api_key(quiet = FALSE)
```

Arguments

api_key	character
verbose	logical, if TRUE the function display a warning if the Api Key was not found!

Value

a character, if the api key was added in the "cryptocompare" environment, otherwise NULL.

Examples

```
# Add the api key to the "cryptocompare" environment
yourapikey <- "yourapikey"
cc_set_api_key(api_key = yourapikey)
```

```
cc_set_environment      cc_set_environment
```

Description

initialize the environment called "cryptocompare" in which are saved information such as the coins ID, the api key and the exchanges names.

Usage

```
cc_set_environment(quiet = FALSE)
```

```
cc_social_historical    cc_social_historical
```

Description

```
cc_social_historical
```

Usage

```
cc_social_historical(
  symbol = NULL,
  start = NULL,
  end = NULL,
  interval = c("daily", "hourly"),
  api_key = NULL
)
```

Arguments

symbol	character, the symbol of interest.
start	character or Date, the start date for importation.
end	character or Date, the end date for importation.
interval	character, the frequency of the data, it can be "daily" or "hourly".
api_key	character

Value

a tibble

cc_symbol_id	<i>cc_symbol_id</i>
--------------	---------------------

Description

retrieve the Id using the symbol, useful for uniforming the social function with the others.

Usage

```
cc_symbol_id(symbol = NULL)
```

Arguments

symbol	character, the symbol for which we would like to know the Id.
--------	---

Value

a character, the symbol ID, if it was found in the "cryptocompare\$coins_Id" dataset, otherwise NULL.

Examples

```
cc_symbol_id(symbol = "BTC")
cc_symbol_id(symbol = "ETH")
```

cc_symbol_price	<i>Symbols Price</i>
-----------------	----------------------

Description

Get the current price of any cryptocurrency in any other currency that you need.

Usage

```
cc_symbol_price(symbol = NULL,
                 exchange = NULL,
                 currency = "USD",
                 api_key = NULL)
```

Arguments

symbol	Character vector. The cryptocurrency symbol of interest, e.g. "BTC", "ETH", "BNB". Default is NULL.
currency	Character vector. Currency symbols to convert into, e.g. "USD", "USDT", "BNB", "EUR". If NULL, the default, will be used "USD".
exchange	Character vector. The exchange to obtain data from. If NULL, the default, will be returned a global mean between all the exchanges.
api_key	character
quiet	Logical. Default is FALSE. If TRUE suppress messages and warnings.

Details

If the crypto does not trade directly into the symbol requested, BTC will be used for conversion. If the opposite pair trades we invert it, e.g. BTC-XMR.

Value

Return a tibble. The number of columns will depends on the lenght of currency parameter. There are 3 columns always present:

- date: [POSIXt](#), time of the snapshot.
- exchange: Character, reference exchange. If exchange is NULL will be classified as Global.
- symbol: Character, tcurrency symbol of interest.

Examples

```
# Mean price for BTCUSD between all the exchanges
cc_symbol_price(symbol = "BTC", exchange = NULL, currency = "USD")

# Mean price for BTCUSD and ETHUSD between all the exchanges
cc_symbol_price(symbol = c("BTC", "ETH"), exchange = NULL, currency = "USD")

# Mean price for BTCUSDT, ETHUSDT on Binance
cc_symbol_price(symbol = c("BTC", "ETH"),
                 exchange = "Binance",
                 currency = c("USDT"))
```

cryptocompare_api	<i>GET call to cryptocompare API</i>
-------------------	--------------------------------------

Description

Execute a GET call to cryptocompare API.

Usage

```
cryptocompare_api(path = NULL, query = NULL, config = list())
```

Arguments

path	Character vector. API path, NULL or NA elements will be excluded.
query	Named list. Query parameters for the API call, NULL or NA elements will be excluded.
config	configuration parameter config from the function GET .

Examples

```
api_path = c("data", "exchange", "histoday")
api_query = list(tsym = "USD",
                 e = "Binance",
                 toTs = as_unix("2023-01-01"),
                 limit = 2000, api_key = NULL)

cryptocompare_api(path = api_path, query = api_query)
```

EMA	<i>SMA</i>
-----	------------

Description

compute the Exponential Moving Average of a vector.

Usage

```
EMA(x = NULL, n = 10, smooth = 2, na.rm = TRUE)
```

Arguments

x	numeric vector.
n	integer, the number of length of the moving average.
smooth	numeric, a parameter to use for smoothing the moving average.
na.rm	logical,

Value

a vector of the same length of x.

SMA	<i>SMA</i>
-----	------------

Description

compute the Simple Moving Average of a vector.

Usage

```
SMA(x = NULL, n = 10, .fun = mean, na.rm = TRUE)
```

Arguments

x	numeric vector.
n	integer, the number of length of the moving average.
.fun	function, the function to use, for example mean, var, sd, ...
na.rm	logical,

Value

a vector of the same length of x .

Index

`add_EMA`, [2](#)
`add_EMA.data.frame`, [2](#)
`add_SMA.data.frame`
 (`add_EMA.data.frame`), [2](#)
`as_unix`, [3](#)

`cc_all_exchanges`, [4](#)
`cc_api_key (cc_set_api_key)`, [12](#)
`cc_blockchain_balance`, [4](#)
`cc_blockchain_historical`, [5](#)
`cc_eth_staking`, [6](#)
`cc_exchange_historical`, [7](#)
`cc_general_info`, [8](#)
`cc_L1_order_book`, [9](#)
`cc_order_book_L2`, [10](#)
`cc_orderbook_exchange`, [10](#)
`cc_price_historical`, [11](#)
`cc_rate_limit`, [12](#)
`cc_set_api_key`, [12](#)
`cc_set_environment`, [13](#)
`cc_social_historical`, [13](#)
`cc_symbol_id`, [14](#)
`cc_symbol_price`, [14](#)
`cryptocompare_api`, [15](#)

`EMA`, [16](#)

`GET`, [15](#)

`POSIXt`, [15](#)

`SMA`, [16](#)